GREEN SOFTWARE ENGINEERING · TESTO COMPLETO DEL TALK

Due sviluppatori, lo stesso task, due impatti diversi

Una storia di scelte quotidiane nello sviluppo software e di uso consapevole dell'AI.

Discorso di circa 30 minuti per un pubblico di sviluppatrici e sviluppatori, da junior a senior. Testo da pronunciare ad alta voce, con tempi, pause, domande e indicazioni di scena.

> **La frase guida**La differenza non è quanta AI usi. È nelle decisioni che prendi attorno a essa: quando serve, quanta ne serve e quanto costa.

Il percorso · 30 minuti

1. Apertura: Anna e Marco — 0-4
2. Contesto: il costo di una richiesta — 4-8
3. Decisione 1: serve davvero l'AI? — 8-11:24
4. Decisione 2: quale modello? — 11:24-14:48
5. Decisione 3: quanto testo? — 14:48-18:12
6. Decisione 4: quante volte? — 18:12-21:36
7. Decisione 5: so quanto costa? — 21:36-25
8. Che cosa distingue Anna e Marco — 25-28
9. Chiusura — 28-30

I tempi includono le pause e le brevi domande al pubblico. Non ci sono numeri di consumo per singola richiesta: le stime dipendono dal modello, dal servizio e da ciò che viene incluso nel calcolo.

Apertura · Due sviluppatori, lo stesso task

0-4 minuti

[PAUSA] [SLIDE: due ticket identici, con i nomi Anna e Marco]

Immaginate due ticket uguali, aperti nello stesso team. Stesso testo, stessa scadenza, stesso cliente. Il task dice: «Classificare automaticamente i messaggi in arrivo dai clienti».

Arriva, per esempio, questo messaggio: «Non riesco a entrare nel mio account». Il sistema dovrebbe riconoscere che è una richiesta di supporto tecnico. Un altro messaggio chiede informazioni su una fattura. Un altro ancora vuole sapere quando arriverà un ordine.

Due sviluppatori prendono in carico il lavoro. Chiamiamoli Anna e Marco.

Marco apre il progetto, guarda le API disponibili, prova un modello AI e nel giro di poco ha una demo che classifica i messaggi. La risposta sembra buona. Il task viene completato. Marco è una persona competente: ha scelto uno strumento che conosce, ha consegnato in tempo e ha fatto funzionare la feature.

Anna riceve lo stesso task. Anche lei valuta un modello AI. Ma prima apre alcuni messaggi di esempio e si chiede che cosa voglia dire davvero «classificare». Le categorie sono tre? Sono dieci? Le regole cambiano spesso? Se arriva un messaggio nuovo, chi lo controlla? Anna alla fine consegna una feature che classifica i messaggi, proprio come Marco.

Da fuori, il risultato è lo stesso: i messaggi arrivano, vengono classificati e il cliente può usarli. Due task completati. Due sviluppatori che hanno fatto il proprio lavoro.

[PAUSA] [DOMANDA AL PUBBLICO: se il risultato è lo stesso, dove sta la differenza?]

Se tutto funziona, dove sta la differenza? Non è nel fatto che Marco abbia usato l'AI e Anna no. Potrebbero averla usata entrambi. E non è che Anna sia una santa del software, o che Marco sia uno sprovveduto. Siamo tutti passati da quel momento: abbiamo un martello nuovo, e per qualche minuto ogni problema ci sembra un chiodo.

La differenza è nelle decisioni che prendiamo prima e dopo aver aperto lo strumento. Serve davvero l'AI? Quale modello basta? Quanta informazione dobbiamo mandare? Quante volte ripetiamo il lavoro? E sappiamo quanto ci costa la feature una volta che è in produzione?

Non conta quanta AI usi. Conta come decidi.

Questa è la storia che useremo per i prossimi trenta minuti. Non per assegnare medaglie ad Anna e colpe a Marco. Ci interessa riconoscere un comportamento comune e vedere come possiamo renderlo un po' più consapevole, da qualunque livello partiamo.

> *Ok, abbiamo due soluzioni che funzionano. Prima di entrare nei dettagli del codice, mettiamo a fuoco una cosa: perché una decisione di design può diventare una decisione ambientale?*

Contesto minimo · Una richiesta è lavoro

4-8 minuti

[SLIDE: testo inviato → modello → testo generato → computer]

Quando mandiamo un messaggio a un modello AI, non parte una magia gratuita dentro lo schermo. Il servizio riceve informazioni, fa calcoli e produce una risposta. In molti modelli di linguaggio, il testo viene diviso in pezzi chiamati token. Pensateli come mattoncini: il modello legge i mattoncini che gli mandiamo e ne costruisce altri per comporre la risposta.

Ogni token generato richiede calcolo. E il calcolo avviene su computer: computer che usano elettricità, producono calore e hanno bisogno di essere raffreddati. Il raffreddamento può coinvolgere acqua, direttamente o indirettamente. Le emissioni di CO_2 dipendono anche da come è stata prodotta l'elettricità che alimenta quei computer.

Questa descrizione non significa che ogni singola richiesta abbia un impatto enorme. Una richiesta individuale può essere piccola. Il punto importante è che un prodotto non riceve una sola richiesta: riceve richieste oggi, domani, la settimana prossima, da tante persone e da tante funzioni diverse.

Immaginiamo la feature di Anna e Marco in un servizio di assistenza. Ogni nuovo messaggio viene classificato. Se il sistema manda una richiesta al modello per ogni messaggio, il lavoro si ripete. Se aggiungiamo un riepilogo automatico, poi un controllo di tono, poi una seconda chiamata per correggere la prima, il task non è più una singola richiesta: è una piccola catena di calcoli.

Non serve spaventarsi di fronte a questa catena. Serve vederla. Il costo ambientale non arriva da una decisione astratta, separata dal nostro lavoro. È legato alle scelte che facciamo nel disegno della feature: il modello che scegliamo, il testo che inviamo, il numero di chiamate e il modo in cui gestiamo gli errori.

Se la feature cresce, il totale cresce con lei. Non perché una richiesta sia improvvisamente diventata cattiva, ma perché ripetiamo un'attività più spesso. E chi scrive il software contribuisce a decidere quante volte la ripetiamo.

> **Tre parole da tenere insieme**
>
> Più calcolo, più energia; l'acqua e le emissioni dipendono anche dal luogo e dall'infrastruttura.

Non abbiamo bisogno di fingere di conoscere il consumo esatto di un singolo messaggio. Le stime variano molto in base al modello, al fornitore e ai confini del calcolo. Se vorremo citare un valore, lo verificheremo e diremo che cosa misura. Per il nostro ragionamento, ci basta questa idea: una chiamata richiede lavoro, e il design decide quanto lavoro chiediamo al sistema.

Ok, abbiamo visto che il design crea lavoro. Ora seguiamo Anna e Marco attraverso cinque decisioni molto pratiche. Per ognuna, una domanda semplice: che cosa facciamo, che cosa possiamo evitare e come capiamo se la scelta regge?

Decisione 1 · Serve davvero l'AI?

8-11:24 · circa 3 minuti e 20

[SLIDE: tre strade — regola, classificatore semplice, modello generativo]

Torniamo ai messaggi in arrivo. La prima decisione non è quale modello usare. È se ci serve un modello, punto.

Marco apre gli esempi. Nota che una parte dei messaggi contiene già un'etichetta scelta dall'utente: «fattura», «accesso», «consegna». C'è anche un modello AI integrabile nel servizio. Marco pensa: se il modello sa capire il testo, farà un buon lavoro. Lo collega alla feature, fa qualche prova e la demo riesce. Non è una scelta assurda. È una scelta rapida e plausibile, soprattutto se il team sta già usando quell'API.

Anna guarda gli stessi messaggi, ma comincia dalle categorie. Se l'utente sceglie una voce da un elenco, non serve chiedere a un modello di indovinarla: possiamo salvare direttamente quella scelta. Per alcuni casi ci sono regole chiare, come riconoscere un oggetto di email standard. Per i messaggi davvero ambigui, si può valutare un classificatore più semplice o un modello linguistico.

La differenza non è «regole buone, AI cattiva». È usare lo strumento proporzionato al problema. Se una regola risolve bene il caso, aggiungere un modello significa aggiungere un servizio da chiamare, una risposta da controllare e un possibile punto di errore.

Un classificatore è un programma che sceglie una categoria tra quelle che gli proponiamo. Pensatelo come un addetto allo smistamento: riceve una lettera e la mette in uno dei contenitori. Un modello generativo, invece, può produrre testo nuovo. Sono capacità diverse; non dobbiamo scegliere quella più potente per abitudine.

[DOMANDA AL PUBBLICO: quali messaggi classifichereste con una regola e quali richiedono davvero di capire il testo?]

Qui la domanda di prodotto conta molto. Se un messaggio deve finire in una coda di lavoro, possiamo anche mostrare «non sono sicuro» e chiedere a una persona di scegliere. A volte la soluzione migliore è un sistema misto: regole per i casi chiari, modello per quelli incerti, controllo umano per i casi importanti.

Da sviluppatori backend sappiamo che il flusso ha già un posto in cui mettere queste decisioni. Possiamo implementare una regola nel codice, una coda di revisione o un confine chiaro prima di chiamare un servizio esterno. Non serve aggiungere una nuova astrazione solo perché il problema contiene testo.

Per Anna, quindi, il primo esperimento è chiedersi quanto spesso la regola risolve il caso e quanti messaggi restano ambigui. Marco non ha sbagliato a provare il modello. Ma se non misura nemmeno quanti casi richiedono davvero l'AI, rischia di far passare dal modello anche i casi già risolti.

> **La frase da ricordare**
>
> «Prima definisci il problema. Poi scegli lo strumento.»

Ecco dove vive in pratica: nella validazione dell'input, nel punto in cui decidiamo quale percorso seguire e nella possibilità di deviare verso un controllo umano. In Symfony, per esempio, Validator può controllare che categoria e messaggio rispettino le regole previste prima di coinvolgere altri servizi. Il nome del componente non è il punto; il punto è mettere la decisione al posto giusto.

Abbiamo deciso quando l'AI serve. La domanda successiva è: quando serve, quanto potente deve essere il modello?

Decisione 2 · Quale modello?

11:24-14:48 · circa 3 minuti e 20

[SLIDE: due modelli, lo stesso test di qualità]

Marco vuole essere sicuro che il sistema riconosca bene le categorie. Guarda i modelli disponibili e sceglie quello più capace, quello che gli sembra più affidabile. È comprensibile: in produzione, nessuno vuole che un cliente scriva «non riesco a entrare» e riceva l'etichetta «consegna». «Prendo il più grande e sto tranquillo», si dice.

Ma scegliere il modello più grande non garantisce automaticamente una risposta migliore per il nostro caso. E non garantisce che la feature sia più affidabile. Se il modello restituisce una risposta convincente ma sbagliata, abbiamo comunque un problema; magari un problema più costoso da individuare.

Anna fa una cosa più noiosa, e proprio per questo utile. Prepara un piccolo gruppo di messaggi rappresentativi: accesso, fattura, consegna, richieste ambigue, testo scritto male. Decide prima che cosa considera una classificazione corretta. Poi prova più di un modello, partendo anche da quello più piccolo e meno costoso che il team può usare.

«Meno costoso» qui può voler dire costo economico più basso o meno calcolo richiesto, ma non traduciamo automaticamente una tariffa in emissioni. Sono grandezze legate, non identiche: le emissioni dipendono anche da dove gira il modello e dall'elettricità usata. La scelta pratica resta comunque misurabile: proviamo modelli diversi sullo stesso compito e osserviamo qualità, tempo e costo.

Se il modello più piccolo raggiunge la qualità che ci serve, usare quello più grande «per stare tranquilli» può significare pagare di più senza ottenere un vantaggio utile. Se invece sbaglia molto, il modello più piccolo non è una virtù: produce correzioni, lavoro manuale e magari nuove chiamate. Un sistema efficiente non è semplicemente quello che consuma meno per tentativo; è quello che completa bene il lavoro senza giri inutili.

La parola chiave è test. Invece di fidarci dell'impressione dopo due esempi, prepariamo casi di prova, includiamo messaggi difficili e ripetiamo il confronto quando cambiano le categorie o il modello. Non serve partire con un enorme processo di valutazione. Serve un test ripetibile, abbastanza vicino ai messaggi reali.

Anna non sceglie il modello più piccolo a priori. Sceglie il più piccolo che supera una soglia di qualità definita per quella feature. Se non supera la soglia, sale di un gradino. E se nessun modello è abbastanza affidabile, il software deve saperlo dire e passare la decisione a una persona.

> **La frase da ricordare**
>
> «Scegli il modello più piccolo che supera i tuoi test.»

Per un team backend, questi test possono stare nella pipeline di integrazione continua, cioè nei controlli automatici che eseguiamo quando il codice cambia. Un aggiornamento del modello non dovrebbe essere trattato come un dettaglio invisibile se cambia la qualità, il tempo di risposta o il costo del nostro endpoint.

Ok, abbiamo scelto la capacità adatta. Ma anche il modello giusto può fare lavoro in più se gli mandiamo un romanzo quando gli basta un paragrafo.

Decisione 3 · Quanto testo?

14:48-18:12 · circa 3 minuti e 20

[SLIDE: richiesta breve e mirata / archivio intero e risposta libera]

Per classificare un messaggio, il modello deve sapere che cosa è arrivato e quali categorie può restituire. Ma nei sistemi reali la richiesta spesso contiene molto altro: istruzioni lunghe, storico della conversazione, documentazione, esempi e qualche messaggio aggiunto «per sicurezza».

Marco costruisce un prompt completo. Include le istruzioni del team, la descrizione del prodotto, l'intero storico del cliente e una richiesta di spiegare in dettaglio il ragionamento. Per ricevere una risposta flessibile, lascia il formato libero. Il sistema funziona, e la spiegazione sembra rassicurante. Però la feature voleva una categoria, non una consulenza di tre pagine.

Anna parte dall'output che serve davvero al codice. La feature ha bisogno di una categoria tra alcune opzioni e, forse, di un livello di incertezza. Anna passa al modello il messaggio corrente e le poche istruzioni necessarie. Se un messaggio precedente è rilevante, lo aggiunge; se non lo è, non invia lo storico completo. Chiede una risposta breve e con un formato definito.

Un formato strutturato è come un modulo con caselle: invece di ricevere una lettera libera, riceviamo campi che il programma sa leggere. Per esempio, categoria e confidenza. Il backend può poi validare che la categoria esista davvero. Se la risposta non rispetta il formato, la rifiuta o la manda in revisione invece di indovinare che cosa intendesse il modello.

Accorciare non significa togliere contesto utile. Se il messaggio è «non funziona», forse senza sapere a quale prodotto si riferisce il modello non può classificare bene. Il criterio non è «meno testo a ogni costo». È: inviare le informazioni sufficienti per fare bene il lavoro, non tutte quelle che abbiamo a disposizione.

Possiamo fare un esperimento molto concreto. Prendiamo gli stessi messaggi e confrontiamo una richiesta piena di contesto con una richiesta essenziale. Controlliamo se la classificazione resta corretta. Se resta corretta e la risposta è più breve, abbiamo tolto lavoro che non contribuiva al risultato. Se peggiora, abbiamo potato troppo e dobbiamo ripristinare il contesto necessario.

C'è anche un tema di sicurezza e riservatezza. Mandare meno dati può ridurre l'esposizione di informazioni personali o interne. Non è una garanzia automatica di privacy, e dobbiamo comunque rispettare le regole del servizio. Ma è una buona abitudine di progettazione: non trasmettere dati che la funzione non usa.

Qui il modello aiuta a costruire il prompt, ma poi noi osserviamo la risposta nella forma in cui il software la consumerà. Non basta che il testo «suoni bene». Deve essere valido, limitato e utile alla feature.

> **La frase da ricordare**
>
> «Manda il contesto che serve, non il contesto che hai.»

Nel backend, il punto pratico può essere una struttura di richiesta costruita con campi espliciti e una risposta controllata con Validator. Anche qui, il componente è solo un esempio: la regola è delimitare ingresso e uscita come faremmo con qualsiasi dipendenza esterna.

Abbiamo ridotto il lavoro di una chiamata. Ora chiediamoci quante volte la facciamo, soprattutto quando la rete o il modello non rispondono come speravamo.

Decisione 4 · Quante volte?

18:12-21:36 · circa 3 minuti e 20

[SLIDE: una richiesta / richieste duplicate / tentativi senza limite]

Il messaggio arriva al server. Marco chiama il modello. Se la chiamata fallisce, il sistema la riprova. Se il client ripete la richiesta perché non ha ricevuto risposta, il server potrebbe chiamare di nuovo. Se un processo automatico continua a vedere un risultato non valido, prova ancora. Ogni singolo meccanismo ha una ragione. Insieme, però, possono trasformare un messaggio in molte chiamate.

Questo è un problema che chi lavora nel backend conosce bene anche senza AI. Una richiesta ripetuta può essere innocua per una lettura nel database, ma non sempre è innocua quando attiva un servizio esterno a pagamento e intensivo di calcolo. Dobbiamo sapere quali operazioni sono ripetibili e dove fermarci.

Anna controlla prima se sta classificando lo stesso messaggio due volte. Se la risposta è già valida e il messaggio non è cambiato, può riutilizzarla. Una cache è una memoria temporanea: come mettere da parte una risposta utile per non rifare subito lo stesso lavoro. Ma la cache funziona solo se sappiamo quando il risultato diventa vecchio e non la usiamo per dati che richiedono una risposta nuova.

Per i messaggi in arrivo, Anna può anche raggruppare il lavoro. Un batch è un gruppo di elementi elaborati insieme invece di uno per volta. È come portare più lettere allo sportello in una sola consegna. Può ridurre la ripetizione di certi passaggi, ma non è adatto a tutto: se il cliente aspetta una risposta immediata, raggruppare troppo può peggiorare l'esperienza.

Poi ci sono i tentativi automatici, i retry. Sono utili quando un servizio ha un problema temporaneo. Ma devono avere un numero massimo, una pausa tra i tentativi e una condizione chiara per smettere. Altrimenti il nostro software può continuare a chiedere proprio quando il servizio è già sotto pressione.

Un rate limit mette un limite alle richieste in un certo periodo. È come un rubinetto: evita che un picco di traffico apra il flusso al massimo senza controllo. Non serve soltanto a proteggere il fornitore. Aiuta anche noi a non moltiplicare per errore il lavoro e il costo.

In Symfony questi punti hanno nomi familiari: Cache per riutilizzare un risultato valido, Messenger per spostare un lavoro non urgente in una coda, RateLimiter per controllare la frequenza. La coda permette di elaborare messaggi in modo ordinato e, se il caso lo consente, raggrupparli o gestirli con priorità. Ma non è un invito a introdurre tutti i componenti in ogni progetto. Prima capiamo il flusso, poi scegliamo l'attrezzo.

Marco potrebbe applicare retry, cache e batch e avere ancora una soluzione sbagliata se riusa una classificazione non più valida o fa aspettare troppo il cliente. Anna non li aggiunge per collezionare pattern. Li usa solo quando corrispondono a una ripetizione reale e ne verifica gli effetti.

> **La frase da ricordare**
>
> «Evita le chiamate duplicate e metti un limite a quelle necessarie.»

Abbiamo scelto una chiamata più snella e ridotto le ripetizioni. Resta una domanda che spesso scopriamo tardi: sappiamo quanto lavoro stiamo chiedendo davvero?

Decisione 5 · So quanto costa?

21:36-25 minuti

[SLIDE: costo e risorse per feature / fattura totale a fine mese]

Marco rilascia la feature. Tutto funziona. Il team si concentra sulle prossime attività e per un po' nessuno guarda da vicino il consumo delle classificazioni. Poi arriva la fattura del servizio, o un alert sul volume di richieste, e la domanda diventa: «Da dove è arrivato tutto questo?».

La fattura ci dà un dato economico utile, ma non sempre ci spiega il costo ambientale. E un contatore di token non ci dà da solo le emissioni. Però misurare l'attività per feature ci aiuta a capire dove stiamo chiedendo lavoro e come cambia nel tempo. Non è necessario conoscere il consumo esatto di ogni server del fornitore per iniziare a fare osservabilità.

Anna aggiunge pochi contatori mirati. Per esempio: quante classificazioni sono state richieste per la feature, quanti token in ingresso e in uscita sono stati usati, quante richieste hanno avuto bisogno di un retry e quante sono finite alla revisione umana. Non raccoglie dati personali nel log se non servono per lo scopo. Misura il comportamento del servizio, non il contenuto privato dei messaggi.

Il team guarda questi valori insieme alla qualità. Se i token aumentano perché la feature ora invia lo storico completo, possiamo scoprirlo. Se il numero di retry cresce, possiamo cercare un problema nel formato o nella disponibilità del servizio. Se le classificazioni aumentano perché la funzione viene usata più spesso, possiamo separare la crescita reale da un bug che duplica le chiamate.

Un budget è un limite o un obiettivo concordato. Possiamo definire un tetto economico per l'uso del servizio, un limite massimo di token per richiesta o una soglia di volume che attiva un avviso. Non sono tutti la stessa misura, ma insieme ci danno un controllo più concreto del «ci penseremo quando arriva la fattura».

In un progetto Symfony, i contatori possono stare accanto alle metriche già raccolte dal backend. L'importante è poter attribuire il lavoro alla feature o al tipo di operazione, senza mettere nei log il prompt completo per comodità. Se non abbiamo accesso ai dati energetici del fornitore, lo diciamo. Possiamo chiedere informazioni, consultare la sua metodologia e fare una stima dichiarandone i limiti, ma non fingere una precisione che non abbiamo.

È utile confrontare nel tempo il costo per lavoro completato: per esempio, quanto costa classificare correttamente un messaggio, non solo quanto costa inviare una chiamata. Se un modello economico sbaglia spesso, richiede revisione e nuove richieste, il costo finale può cambiare. E se il servizio cresce, il costo unitario e il costo totale raccontano due storie diverse: monitoriamo entrambi.

Marco non è negligente perché all'inizio non aveva tutto questo. È normale che una demo misuri soprattutto se la funzione funziona. La decisione matura è aggiungere osservabilità prima che il comportamento diventi invisibile in produzione.

> **La frase da ricordare**
> «Misura per feature, fissa un budget e guarda il costo per risultato corretto.»

Ok, abbiamo visto cinque decisioni: quando usare l'AI, quale modello, quanto testo, quante chiamate e quanto costa. Ora possiamo tornare ad Anna e Marco. Che cosa li distingue davvero?

Che cosa distingue davvero Anna e Marco

25-28 minuti

[SLIDE: stessa feature, cinque domande di design]

Non la seniority. Non il numero di strumenti nel curriculum. Non il fatto che Anna abbia detto sempre no e Marco sempre sì. La differenza è il giudizio: il momento in cui ci fermiamo, descriviamo il bisogno e scegliamo quanto lavoro chiedere al sistema.

Marco rappresenta una scorciatoia che conosciamo. Abbiamo una capacità nuova, la vediamo funzionare in una demo e la applichiamo anche ai casi vicini. È comprensibile. Ci aiuta a imparare, a consegnare e a scoprire cose che non avevamo considerato. Il problema nasce se il prototipo diventa la decisione definitiva senza che nessuno si chieda se quella capacità serva a ogni passaggio.

C'è poi un paradosso curioso. Chiediamo all'AI come costruire una feature, e spesso ci propone... più AI. «Aggiungi un agente che controlla il risultato. Poi un secondo agente che verifica il primo. E magari un terzo che riassume la conversazione». [SORRIDI] È una risposta plausibile dal punto di vista del testo: il modello continua la soluzione che gli sembra più probabile. Ma non è lui a pagare la fattura, a gestire i retry o a rispondere del servizio alle tre di notte.

Quella parte è nostra. L'AI può suggerire una strada; lo sviluppatore deve chiedere se è la strada necessaria. A volte la risposta migliore al suggerimento del modello è: «Grazie, ma qui basta una condizione nel codice».

Questo istinto va costruito, soprattutto da chi sta iniziando adesso. Se impariamo a programmare in un momento in cui un modello può generare codice, è facile confondere il saper chiedere con il saper progettare. Chiedere è utile. Progettare significa anche stabilire i confini: cosa deleghiamo, che cosa verifichiamo e quando non deleghiamo niente.

Per questo non voglio dire che dobbiamo usare meno AI per principio. Voglio dire che possiamo usare AI dove porta valore, e non usarla dove un metodo più semplice raggiunge lo stesso risultato. È una forma di rispetto per il cliente, per chi mantiene il software e per le risorse che il servizio consuma.

Il costo ambientale e quello economico spesso si muovono nella stessa direzione: chiamate superflue, modelli sovradimensionati e contesto ripetuto tendono a richiedere più risorse e a costare di più. Non sono perfettamente identici. Il prezzo dipende dal contratto e l'impatto ambientale dipende dall'infrastruttura, dall'energia e dal luogo. Ma le decisioni di efficienza sono spesso una buona domanda per entrambi: stiamo pagando, in denaro e in calcolo, per un lavoro che non migliora il risultato?

Anna e Marco consegnano entrambi. La differenza è che Anna rende visibili le ipotesi e sa spiegare perché ogni chiamata c'è. Marco può arrivarci allo stesso modo: non deve buttare via la feature. Può aprire un ticket di follow-up, aggiungere una misura, confrontare un modello più piccolo e togliere i passaggi che non aggiungono qualità.

Non conta quanta AI usi. Conta come decidi.

> *Ritorniamo alla domanda iniziale: se il risultato è lo stesso, dove sta la differenza? Adesso possiamo dirlo in modo concreto.*

Chiusura · La decisione è parte del codice

28-30 minuti

[PAUSA] [SLIDE: stesso task, le cinque decisioni, una frase finale]

Due sviluppatori, lo stesso task, due impatti diversi. La differenza non è che uno ha usato l'AI e l'altro l'ha rifiutata. La differenza è nelle decisioni attorno alla feature: se l'AI serviva, quale modello bastava, quanto testo mandare, quante volte chiamare il servizio e come tenere sotto controllo il costo.

Una richiesta può pesare poco. Ma il design decide se quella richiesta resta una, se diventa dieci tentativi, se include un intero archivio e se viene ripetuta per ogni messaggio anche quando il risultato era già disponibile.

La prossima volta che lavorate su una feature AI, fate una cosa semplice. Nel ticket, aggiungete una riga: perché abbiamo scelto questo modello? Oppure: perché qui non abbiamo scelto un modello? Se la risposta è chiara, avete reso la decisione visibile a chi rivedrà, manterrà e userà il software.

[DOMANDA AL PUBBLICO: qual è la prima delle cinque decisioni che porterete nel prossimo ticket?]

Un buon sviluppatore non è quello che usa più AI, ma quello che sa quanta ne serve.

[PAUSA] Grazie.

Scaletta · una pagina

Filo: stesso task → cinque decisioni → giudizio → azione nel ticket.

1. **0-4 · Apertura:** due ticket identici. Marco e Anna consegnano. Domanda: se il risultato è lo stesso, dove sta la differenza?
2. **4-8 · Contesto:** token = mattoncini di testo; generare token richiede calcolo; calcolo richiede energia. Acqua e CO_2 dipendono anche da raffreddamento, luogo e mix elettrico. Una richiesta può essere piccola; conta la somma decisa dal design.
3. **8-11:24 · Serve l'AI?** Regola → classificatore semplice → modello generativo, solo se serve. *Definisci il problema, poi scegli lo strumento.*
4. **11:24-14:48 · Quale modello?** Test sullo stesso gruppo di messaggi; qualità, tempo, costo. Parti dal più piccolo che supera la soglia. *Scegli il modello più piccolo che supera i tuoi test.*
5. **14:48-18:12 · Quanto testo?** Contesto necessario, prompt mirato, output limitato e validato. *Manda il contesto che serve, non il contesto che hai.*
6. **18:12-21:36 · Quante volte?** Cache valida, batch quando adatto, retry limitati, rate limit. Symfony: Cache, Messenger, RateLimiter. *Evita le chiamate duplicate e metti un limite a quelle necessarie.*
7. **21:36-25 · Quanto costa?** Token e chiamate per feature, qualità, retry, budget; distinguere fattura, token ed emissioni. Symfony/metriche di backend. *Misura per feature, fissa un budget e guarda il costo per risultato corretto.*
8. **25-28 · Giudizio:** Anna non è santa, Marco non è sprovveduto. L'AI può suggerire più AI: a noi spetta dire «qui non serve». I costi economici e ambientali spesso si muovono insieme, ma non sono la stessa misura.
9. **28-30 · Chiusura:** ritorno alla domanda; scrivere nel ticket perché si è scelto (o non scelto) un modello. Frase finale: «Un buon sviluppatore non è quello che usa più AI, ma quello che sa quanta ne serve.»

Le 5 frasi da ricordare parola per parola

1. Non conta quanta AI usi. Conta come decidi.
2. Prima definisci il problema. Poi scegli lo strumento.
3. Scegli il modello più piccolo che supera i tuoi test.
4. Manda il contesto che serve, non il contesto che hai.
5. Un buon sviluppatore non è quello che usa più AI, ma quello che sa quanta ne serve.

Tre consigli per memorizzarlo

1. **Impara il percorso, non ogni virgola.** Ricorda apertura, contesto, cinque domande, giudizio e azione finale. Le frasi-slogan fanno da appigli.
2. **Ripeti a blocchi.** Prova prima una sezione alla volta; poi aggiungi la transizione verso la successiva. Usa le stesse parole per i cinque titoli.
3. **Simula ad alta voce con le slide.** Cronometrati, registra una prova e ascolta dove acceleri. Se sfori, taglia un esempio, non il filo del discorso.

Dati da verificare prima del talk

- **[DATO DA VERIFICARE: eventuale stima del consumo energetico per richiesta]** Da citare solo se serve all'argomento; indicare modello, tipo di richiesta, metodo, data, confini e fonte primaria. Non esiste un valore universale per ogni richiesta.
- **[DATO DA VERIFICARE: eventuali dati su acqua e raffreddamento del fornitore]** Verificare sede e infrastruttura, distinguere uso diretto e indiretto dell'acqua e indicare periodo e metodologia.
- **[DATO DA VERIFICARE: eventuale intensità carbonica dell'elettricità]** Specificare località e periodo; non convertire token in CO_2 senza ipotesi documentate.
- **[DATO DA VERIFICARE: costi e limiti dei modelli in uso]** Controllare prezzi, limiti di token, modalità di fatturazione e data del listino del fornitore.

Nel testo del talk non vengono presentate stime numeriche o fonti non controllate. Se aggiungi dati, aggiorna questa lista e riportali con il loro perimetro.